Contents

1 Basis 1
- 1.1 Default Code . . . 1
- 1.2 Some Useful Primes . . . 1

2 Data Structures 1
- 2.1 Disjoint Set (Union-Find) . . . 1
- 2.2 Binary Indexed Tree . . . 1
- 2.3 Segment Tree . . . 1

3 Mathematics 2
- 3.1 Combination with Inverse . . . 2
- 3.2 EXGCD . . . 2
- 3.3 FFT . . . 2
- 3.4 Exponentiation by Squaring . . . 2
- 3.5 GCD and LCM . . . 2
- 3.6 Miller-Rabin Algorithm . . . 3
- 3.7 Pollard's Rho Algorithm . . . 3

4 Graph 3
- 4.1 Tarjan's Algorithm for BCC . . . 3
- 4.2 Bellmen-Ford Algorithm . . . 3
- 4.3 Dijkstra's Algorithm . . . 3
- 4.4 Dinic's Algorithm for Maximum Flow . . . 4
- 4.5 Dominator Tree . . . 4
- 4.6 Floyd-Warshall Algorithm . . . 5
- 4.7 Maximum Clique . . . 5
- 4.8 Kosaraju's Algorithm for SCC . . . 5

5 Tree Algorithms 5
- 5.1 Lowest Common Ancester . . . 5
- 5.2 Eular Tour for LCA . . . 6
- 5.3 DSU on Tree . . . 6

6 String Algorithms 6
- 6.1 Suffix Array (SAIS) . . . 6
- 6.2 Z-algorithm . . . 7
- 6.3 Knuth-Morris-Pratt Algorithm . . . 7
- 6.4 Manacher's Algorithm . . . 7
- 6.5 Minimum Rotation . . . 8
- 6.6 Rolling Hash . . . 8

7 Geometry 8
- 7.1 Basic Geometry Structure . . . 8
- 7.2 Circle Cover Area . . . 8
- 7.3 Circles' Tangent Lines . . . 9
- 7.4 Circles' Intersections . . . 9
- 7.5 Convex Hull . . . 9
- 7.6 Convex Hull Tricks . . . 10
- 7.7 Pick's Theory . . . 10

8 Sorting 10
- 8.1 Merge Sort . . . 10
- 8.2 Quick Sort . . . 11

9 Other Useful Shit 11
- 9.1 Discretization . . . 11
- 9.2 The Most Useful One . . . 11

1 Basis

1.1 Default Code

```cpp
#include <bits/stdc++.h>
#define REP(i, n) for (int i = 0; i < n; i++)
#define PB push_back
#define SZ(x) (int)x.size()
#define ll long long
#define ld long double
#define pii pair<int, int>
#define INF 0x7f7f7f7f7f7f7f7fLL
#define all(x) x.begin(), x.end()
#define D double
using namespace std;
const int MXN = 2e6 + 5;
const int MOD = 1e9 + 7;
int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie
        (0);
    return 0;
}
```

1.2 Some Useful Primes

```cpp
/* 12721, 13331, 14341, 75577, 123457, 222557, 556679
 * 999983, 1097774749, 1076767633, 100102021, 999997771
 * 1001010013, 1000512343, 987654361, 999991231
 * 999888733, 98789101, 987777733, 999991921,
     1010101333
 * 1010102101, 1000000000039, 1000000000000037
 * 2305843009213693951, 4611686018427387847
 * 9223372036854775783, 18446744073709551557 */
```

2 Data Structures

2.1 Disjoint Set (Union-Find)

```cpp
struct DSU {
    int n;
    vector<int> f, sz;
    DSU(int _n) {  // 初始化
        n = _n;
        f.resize(n);
        sz.resize(n);
        for (int i = 0; i < n; i++) {
            f[i] = i;
            sz[i] = 1;
        }
    }
    int find(int x) {  // 返回 x 的根節點
        if (x == f[x])
            return x;
        return f[x] = find(f[x]);
    }
    void merge(int x, int y) {  // 將 x 和 y 合併
        x = find(x), y = find(y);
        if (x == y)
            return;
        if (sz[x] < sz[y])  // 將小的併入大的
            swap(x, y);
        sz[x] += sz[y];
        f[y] = x;
    }
};
```

2.2 Binary Indexed Tree

```cpp
#define lowbit(x) (x & -x)
struct BIT {
    int n;
    vector<ll> a;
    BIT(int _n) {
        n = _n;
        a.clear();
        a.resize(n + 1, 0);
    }
    void update(int x, int v) {  // 將 x 的值加上 v
        for (; x < a.size(); x += lowbit(x))
            a[x] += v;
    }
    ll query(int x) {  // 查詢區間 [1, x] 的總和
        ll ret = 0;
        for (; x; x -= lowbit(x))
            ret += a[x];
        return ret;
    }
    // 查詢區間 [l, r] 的總和
    ll query(int l, int r) {  // 大多呼叫這個為主
        return query(r) - query(l - 1);
    }
    int kth(int k) {  // 說實話這不常用 有時可以刪掉
        int x = 0;
        for (int i = 1 << __lg(n); i; i >>= 1) {
            if (x + i <= n && k >= a[x + i - 1]) {
                x += i;
                k -= a[x - 1];
            }
        }
        return x;
    }
};
```

2.3 Segment Tree

```
#define cl (i << 1)
#define cr (i << 1 | 1)
#define NO_TAG 0
struct SegmentTree {  // 1-base
    int n;
    vector<int> seg, tag;
    SegmentTree(int _n) {  // 空的 SegmentTree
        n = _n;
        seg.resize(n * 4);
        tag.resize(n * 4);
    }
    SegmentTree(vector<int>& arr) {  // 處理好的
        SegmentTree
        n = arr.size();
        seg.resize(n * 4);
        tag.resize(n * 4);
        build(1, 0, n - 1, arr);  // kk
    }
    void push(int i, int l, int r) {
        if (tag[i] != NO_TAG) {
            seg[i] += tag[i];  // update by tag
            if (l != r) {
                tag[cl] += tag[i];  // push
                tag[cr] += tag[i];  // push
            }
            tag[i] = 0;
        }
    }
    void pull(int i, int l, int r) {
        int mid = (l + r) >> 1;
        push(cl, l, mid);
        push(cr, mid + 1, r);
        seg[i] = max(seg[cl], seg[cr]);  // pull
    }
    void build(int i, int l, int r, vector<int>& arr) {
        if (l == r) {
            seg[i] = arr[l];  // set value
            return;
        }
        int mid = (l + r) >> 1;
        build(cl, l, mid, arr);
        build(cr, mid + 1, r, arr);
        pull(i, l, r);
    }
    void update(int i, int l, int r, int ql, int qr,
        int v) {
        push(i, l, r);
        if (ql <= l && r <= qr) {
            tag[i] += v;
            return;
        }
        int mid = (l + r) >> 1;
        if (ql <= mid)
            update(cl, l, mid, ql, qr, v);
        if (qr > mid)
            update(cr, mid + 1, r, ql, qr, v);
        pull(i, l, r);
    }
    int query(int i, int l, int r, int ql, int qr) {
        push(i, l, r);
        if (ql <= l && r <= qr) {
            return seg[i];
        }
        int mid = (l + r) >> 1, ans = -INF;
        if (ql <= mid)
            ans = max(ans, query(cl, l, mid, ql, qr));
        if (qr > mid)
            ans = max(ans, query(cr, mid + 1, r, ql, qr
                ));
        return ans;
    }
    // 區間 [ql, qr] 加值 v
    void update(int ql, int qr, int v) {
        update(1, 0, n - 1, ql, qr, v);  // kk
    }
    // 查詢區間 [ql, qr]
    int query(int ql, int qr) {
        return query(1, 0, n - 1, ql, qr);  // kk
    }
    // 不知道是 [0, n-1] 還是 [1, n]
};
```

3 Mathematics

3.1 Combination with Inverse

```
ll fac[MXN], inv[MXN];

void init() {
    fac[0] = 1;                    // 0! = 1
    for (ll i = 1; i < MXN; i++)  // factorial
        fac[i] = fac[i - 1] * i % MOD;
    // 費馬小定理(逆元) with 快速冪
    inv[MXN - 1] = power(fac[MXN - 1], MOD - 2);
    for (ll i = MXN - 2; i >= 0; i--)  // the inverse
        of factorial
        inv[i] = inv[i + 1] * (i + 1) % MOD;
}

// 組合 (Combination) n 取 k
ll comb(ll n, ll k) {
    // C(n, k) = n! / (k! * (n - k)!)
    return fac[n] * inv[k] % MOD * inv[n - k] % MOD;
}

// 排列 (Permutation) n 取 k
ll perm(ll n, ll k) {
    // P(n, k) = C(n, k) * k!
    return fac[n] * inv[n - k] % MOD;
}

// 重複組合 (Combinations WITH Repetitions)
ll h(ll n, ll k) {
    // H(n, k) = C(n + k - 1, n - 1)
    return fac[n + k - 1] * inv[k] % MOD * inv[n - 1] %
         MOD;
}
```

3.2 EXGCD

```
int exgcd(int a, int b, long long &x, long long &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    int now = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return now;
}
```

3.3 FFT

```
const int MAXN = 262144 << 1;
// (must be 2^k)
// before any usage, run pre_fft() first
typedef long double ld;
typedef complex<ld> cplx;  // real(), imag()
const ld PI = acosl(-1);
const cplx I(0, 1);
cplx omega[MAXN + 1];
void pre_fft() {
    for (int i = 0; i <= MAXN; i++)
        omega[i] = exp(i * 2 * PI / MAXN * I);
}
// n must be 2^k
void fft(int n, cplx a[], bool inv = false) {
    int basic = MAXN / n;
    int theta = basic;
    for (int m = n; m >= 2; m >>= 1) {
        int mh = m >> 1;
        for (int i = 0; i < mh; i++) {
            cplx w = omega[inv ? MAXN - (i * theta %
                MAXN)
                               : i * theta % MAXN];
            for (int j = i; j < n; j += m) {
                int k = j + mh;
                cplx x = a[j] - a[k];
                a[j] += a[k];
                a[k] = w * x;
            }
        }
```

```
        theta = (theta * 2) % MAXN;
    }
    int i = 0;
    for (int j = 1; j < n - 1; j++) {
        for (int k = n >> 1; k > (i ^= k); k >>= 1) {}
        if (j < i)
            swap(a[i], a[j]);
    }
    if (inv)
        for (i = 0; i < n; i++)
            a[i] /= n;
}
cplx arr[MAXN + 1];
inline void mul(int _n, ll a[], int _m, ll b[], ll ans
    []) {
    int n = 1, sum = _n + _m - 1;
    while (n < sum)
        n <<= 1;
    for (int i = 0; i < n; i++) {
        double x = (i < _n ? a[i] : 0), y = (i < _m ? b
            [i] : 0);
        arr[i] = complex<double>(x + y, x - y);
    }
    fft(n, arr);
    for (int i = 0; i < n; i++)
        arr[i] = arr[i] * arr[i];
    fft(n, arr, true);
    for (int i = 0; i < sum; i++)
        ans[i] = (long long int)(arr[i].real() / 4 +
            0.5);
}
```

3.4 Exponentiation by Squaring

```
ll power(ll x, ll y) {
    ll ret = 1;
    while (y) {
        if (y & 1)
            (ret *= x) %= MOD;
        (x *= x) %= MOD;
        y >>= 1;
    }
    return ret;
}
```

3.5 GCD and LCM

```
ll gcd(ll a, ll b) {
    if (b && a)  // 輾轉先除法->交叉取餘數直到不能再取
        while ((a %= b) && (b %= a)) {}
    return a + b;
}

ll lcm(ll a, ll b) {
    return a * b * gcd(a, b);
}
```

3.6 Miller-Rabin Algorithm

```
ll mul(ll x, ll y, ll mod) {
    ll ret = x * y - (ll)((long double)x / mod * y) *
        mod;
    return ret < 0 ? ret + mod : ret;
    // return x * y % mod;  //// 如果數字大到需要開
        __int128就用這
}

ll magic[] = {};  // 這邊填入要用於測試的數列
ll S = 3;         // 測試的數列數字的數量
bool witness(ll a, ll n, ll u, int t) {
    if (!a)
        return 0;
    ll x = power(a, u, n);
    for (int i = 0; i < t; i++) {
        ll nx = mul(x, x, n);
        if (nx == 1 && x != 1 && x != n - 1)
            return 1;
        x = nx;
    }
    return x != 1;
}
bool miller_rabin(ll n) {
    int s = S;  // magic number size
    // iterate s times of witness on n
    if (n < 2)
        return false;
    if (!(n & 1))
        return (n == 2);

    // 將 n - 1 寫成 u * (2 ^ t) 的形式
    ll u = n - 1;
    int t = 0;
    while (!(u & 1))
        u >>= 1, t++;

    while (s--) {
        ll a = magic[s] % n;
        if (witness(a, n, u, t))
            return false;
    }
    return true;
}
```

3.7 Pollard's Rho Algorithm

```
vector<ll> ret;

ll mul(ll x, ll y, ll mod) {
    ll ret = x * y - (LL)((long double)x / mod * y) *
        mod;
    return ret < 0 ? ret + mod : ret;
    // return x * y % mod;  //// for __int128
}

ll f(ll x, ll c, ll mod) {
    return (mul(x, x, mod) + c) % mod;
}

ll pollard_rho(ll n) {
    ll c = 1, x = 0, y = 0, p = 2, q, t = 0;
    while (t++ % 128 or gcd(p, n) == 1) {
        if (x == y)
            c++, y = f(x = 2, c, n);
        if (q = mul(p, abs(x - y), n))
            p = q;
        x = f(x, c, n);
        y = f(f(y, c, n), c, n);
    }
    return gcd(p, n);
}

void fact(ll x) {  // 透過遞迴找質數
    if (miller_rabin(x)) {
        ret.push_back(x);
        return;
    }
    ll f = pollard_rho(x);
    fact(f);
    fact(x / f);
}
```

4 Graph

4.1 Topological Sort

```
vector<int> edge[MAXN], ans;
int deg[MAXN];      // in degree
void topo(int n) {  // 1-base
    queue<int> que;
    for (int i = 1; i <= n; i++)
        if (!deg[i])
            que.push(i);
    while (!que.empty()) {
        int u = que.front();
        que.pop();
        ans.push_back(u);
        // 結果存 ans
        for (int v : edge[u]) {
            deg[v]--;
            if (!deg[v])
                que.push(v);
        }
    }
```

```
}
```

4.2 Tarjan's Algorithm for BCC

```
struct BccVertex {
    int n, nScc, step, dfn[MXN], low[MXN];
    vector<int> E[MXN], sccv[MXN];
    int top, stk[MXN];
    void init(int _n) {
        n = _n;
        nScc = step = 0;
        for (int i = 0; i < n; i++)
            E[i].clear();
    }
    void addEdge(int u, int v) {
        E[u].PB(v);
        E[v].PB(u);
    }
    void DFS(int u, int f) {
        dfn[u] = low[u] = step++;
        stk[top++] = u;
        for (auto v : E[u]) {
            if (v == f)
                continue;
            if (dfn[v] == -1) {
                DFS(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] >= dfn[u]) {
                    int z;
                    sccv[nScc].clear();
                    do {
                        z = stk[--top];
                        sccv[nScc].PB(z);
                    } while (z != v);
                    sccv[nScc++].PB(u);
                }
            } else
                low[u] = min(low[u], dfn[v]);
        }
    }
    vector<vector<int>> solve() {
        vector<vector<int>> res;
        for (int i = 0; i < n; i++)
            dfn[i] = low[i] = -1;
        for (int i = 0; i < n; i++)
            if (dfn[i] == -1) {
                top = 0;
                DFS(i, i);
            }
        REP(i, nScc) res.PB(sccv[i]);
        return res;
    }
} graph;
```

4.3 Bellmen-Ford Algorithm

```
struct Edge {
    int from, to, weight;
} edge[MAXN];
int dis[MAXN];
void bellmen_ford(int n, int m) {
    dis[1] = 0;                          // 起點的距離一
        定是 0
    for (int j = 0; j < n - 1; j++) {  // n 個節點要跑
        n - 1 次
        for (int i = 0; i < m; i++) {  // 對於所有邊都
            嘗試鬆弛
            if (dis[edge[i].to] > dis[edge[i].from] +
                edge[i].weight)
                dis[edge[i].to] = dis[edge[i].from] +
                    edge[i].weight;
        }
    }
}
```

4.4 Dijkstra's Algorithm

```
vector<pii> vec[N];  // vec[u] = {v, w}: u 為起點 v 為
    終點 w 為路權
bool vis[N];         // 紀錄該節點是否走過
vector<int> dijkstra(int s) {  // 起點
    vector<int> dis(N);
    for (int i = 0; i < N; i++)  // 初始化
        dis[i] = INF;  // 值要設為比可能的最短路徑權重
            還要大的值
    dis[s] = 0;
    priority_queue<pii, vector<pii>, greater<pii>> pq;
        // 以小到大排序
    // {w, v}: w 為路權 v 為節點
    pq.push({dis[s], s});
    while (pq.empty() == 0) {
        int u = pq.top().second;
        pq.pop();
        if (vis[u])  // 走過的不要再走
            continue;
        vis[u] = 1;
        for (auto i : vec[u]) {
            int v = i.first, w = i.second;
            if (dis[u] + w < dis[v]) {  // 鬆弛
                dis[v] = dis[u] + w;
                pq.push({dis[v], v});
            }
        }
    }
    return dis;
}
```

4.5 Dinic's Algorithm for Maximum Flow

```
struct Dinic {
    struct Edge {
        int v, f, re;
    };
    int n, s, t, level[MXN];
    vector<Edge> E[MXN];
    void init(int _n, int _s, int _t) {
        n = _n;
        s = _s;
        t = _t;
        for (int i = 0; i < n; i++)
            E[i].clear();
    }
    void add_edge(int u, int v, int f) {
        E[u].PB({v, f, SZ(E[v])});
        E[v].PB({u, 0, SZ(E[u]) - 1});
    }
    bool BFS() {
        for (int i = 0; i < n; i++)
            level[i] = -1;
        queue<int> que;
        que.push(s);
        level[s] = 0;
        while (!que.empty()) {
            int u = que.front();
            que.pop();
            for (auto it : E[u]) {
                if (it.f > 0 && level[it.v] == -1) {
                    level[it.v] = level[u] + 1;
                    que.push(it.v);
                }
            }
        }
        return level[t] != -1;
    }
    int DFS(int u, int nf) {
        if (u == t)
            return nf;
        int res = 0;
        for (auto &it : E[u]) {
            if (it.f > 0 && level[it.v] == level[u] +
                1) {
                int tf = DFS(it.v, min(nf, it.f));
                res += tf;
                nf -= tf;
                it.f -= tf;
                E[it.v][it.re].f += tf;
                if (nf == 0)
                    return res;
            }
        }
        if (!res)
            level[u] = -1;
```

```
        return res;
    }
    int flow(int res = 0) {
        while (BFS())
            res += DFS(s, 2147483647);
        return res;
    }
} flow;
```

4.6 Dominator Tree

```
#define REP(i, s, e) for (int i = (s); i <= (e); i++)
#define REPD(i, s, e) for (int i = (s); i >= (e); i--)
struct DominatorTree {  // O(N)
    int n, s;
    vector<int> g[MAXN], pred[MAXN];
    vector<int> cov[MAXN];
    int dfn[MAXN], nfd[MAXN], ts;
    int par[MAXN];  // idom[u] s到u的最後一個必經點
    int sdom[MAXN], idom[MAXN];
    int mom[MAXN], mn[MAXN];
    inline bool cmp(int u, int v) {
        return dfn[u] < dfn[v];
    }
    int eval(int u) {
        if (mom[u] == u)
            return u;
        int res = eval(mom[u]);
        if (cmp(sdom[mn[mom[u]]], sdom[mn[u]]))
            mn[u] = mn[mom[u]];
        return mom[u] = res;
    }
    void init(int _n, int _s) {
        ts = 0;
        n = _n;
        s = _s;
        REP(i, 1, n) g[i].clear(), pred[i].clear();
    }
    void addEdge(int u, int v) {
        g[u].push_back(v);
        pred[v].push_back(u);
    }
    void dfs(int u) {
        ts++;
        dfn[u] = ts;
        nfd[ts] = u;
        for (int v : g[u])
            if (dfn[v] == 0) {
                par[v] = u;
                dfs(v);
            }
    }
    void build() {
        REP(i, 1, n) {
            dfn[i] = nfd[i] = 0;
            cov[i].clear();
            mom[i] = mn[i] = sdom[i] = i;
        }
        dfs(s);
        REPD(i, n, 2) {
            int u = nfd[i];
            if (u == 0)
                continue;
            for (int v : pred[u])
                if (dfn[v]) {
                    eval(v);
                    if (cmp(sdom[mn[v]], sdom[u]))
                        sdom[u] = sdom[mn[v]];
                }
            cov[sdom[u]].push_back(u);
            mom[u] = par[u];
            for (int w : cov[par[u]]) {
                eval(w);
                if (cmp(sdom[mn[w]], par[u]))
                    idom[w] = mn[w];
                else
                    idom[w] = par[u];
            }
            cov[par[u]].clear();
        }
        REP(i, 2, n) {
            int u = nfd[i];
            if (u == 0)
                continue;
            if (idom[u] != sdom[u])
                idom[u] = idom[idom[u]];
        }
    }
} domT;
```

4.7 Floyd-Warshall Algorithm

```
int dis[N][N];
void init(int n) {
    for (int i = 0; i <= n; i++) {  // 初始化
        for (int j = 0; j <= n; j++)
            dis[i][j] = INF;
        dis[i][i] = 0;
    }
}
void floyd_warshall(int n) {
    for (int k = 0; k < n; k++) {  // 窮舉中繼點 k
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {  // 窮舉點對(i, j)
                dis[i][j] = min(dis[i][j], dis[i][k] +
                    dis[k][j]);
            }
        }
    }
}
```

4.8 Maximum Clique

```
#define N 111       // 80 ~ 100 左右
struct MaxClique {  // 0-base
    typedef bitset<N> Int;
    Int linkto[N], v[N];
    int n;
    void init(int _n) {
        n = _n;
        for (int i = 0; i < n; i++) {
            linkto[i].reset();
            v[i].reset();
        }
    }
    void addEdge(int a, int b) {
        v[a][b] = v[b][a] = 1;
    }
    int popcount(const Int& val) {
        return val.count();
    }
    int lowbit(const Int& val) {
        return val._Find_first();
    }
    int ans, stk[N];
    int id[N], di[N], deg[N];
    Int cans;
    void maxclique(int elem_num, Int candi) {
        if (elem_num > ans) {
            ans = elem_num;
            cans.reset();
            for (int i = 0; i < elem_num; i++)
                cans[id[stk[i]]] = 1;
        }
        int potential = elem_num + popcount(candi);
        if (potential <= ans)
            return;
        int pivot = lowbit(candi);
        Int smaller_candi = candi & (~linkto[pivot]);
        while (smaller_candi.count() && potential > ans
            ) {
            int next = lowbit(smaller_candi);
            candi[next] = !candi[next];
            smaller_candi[next] = !smaller_candi[next];
            potential--;
            if (next == pivot ||
                (smaller_candi & linkto[next]).count())
                    {
                stk[elem_num] = next;
                maxclique(elem_num + 1, candi & linkto[
                    next]);
            }
        }
```

```
    }
    int solve() {
        for (int i = 0; i < n; i++) {
            id[i] = i;
            deg[i] = v[i].count();
        }
        sort(id, id + n,
             [&](int id1, int id2) { return deg[id1] >
                 deg[id2]; });
        for (int i = 0; i < n; i++)
            di[id[i]] = i;
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (v[i][j])
                    linkto[di[i]][di[j]] = 1;
        Int cand;
        cand.reset();
        for (int i = 0; i < n; i++)
            cand[i] = 1;
        ans = 1;
        cans.reset();
        cans[0] = 1;
        maxclique(0, cand);
        return ans;
    }
} solver;
```

4.9 Kosaraju's Algorithm for SCC

```
#define FZ(x) memset(x, 0, sizeof(x))  // fill zero
struct Scc {
    int n, nScc, vst[MXN], bln[MXN];
    vector<int> E[MXN], rE[MXN], vec;
    void init(int _n) {
        n = _n;
        for (int i = 0; i <= n; i++)
            E[i].clear(), rE[i].clear();
    }
    void addEdge(int u, int v) {
        E[u].PB(v);
        rE[v].PB(u);
    }
    void DFS(int u) {
        vst[u] = 1;
        for (auto v : E[u])
            if (!vst[v])
                DFS(v);
        vec.PB(u);
    }
    void rDFS(int u) {
        vst[u] = 1;
        bln[u] = nScc;  // 編號是 0-base
        for (auto v : rE[u])
            if (!vst[v])
                rDFS(v);
    }
    void solve() {
        nScc = 0;
        vec.clear();
        fill(vst, vst + n + 1, 0);
        for (int i = 0; i < n; i++)
            if (!vst[i])
                DFS(i);
        reverse(vec.begin(), vec.end());
        fill(vst, vst + n + 1, 0);
        for (auto v : vec)
            if (!vst[v]) {
                rDFS(v);
                nScc++;
            }
    }
};
```

5 Tree Algorithms

5.1 Lowest Common Ancester

```
#include <bits/stdc++.h>
using namespace std;

const int N = 2e5 + 5, logN = 20;
vector<int> g[N];

int timing = 1, logn, n;
int in[N], out[N];
int anc[N][logN], depth[N];

// 建立倍增表
void build_table(int n) {
    logn = log2(n);
    for (int i = 1; i < logN; i++) {
        for (int now = 1; now <= n; now++) {
            anc[now][i] = anc[anc[now][i - 1]][i - 1];
        }
    }
}

// 建立時間戳記
// 跑一遍 DFS 得到誰是誰的父親
void dfs_init(int u, int pre) {
    in[u] = timing++;       // 這時進入u
    anc[u][0] = pre;        // now的0倍祖先是pre
    for (int nxt : g[u]) {  // 跑過所有孩子
        if (pre == nxt)
            continue;
        depth[nxt] = depth[u] + 1;
        dfs_init(nxt, u);
    }
    out[u] = timing++;  // 這時離開u
}

// 判斷是否是祖先
bool is_ancestor(int u, int v) {
    return (in[u] < in[v] && out[v] < out[u]);
}

// 找最低共同祖先
int getlca(int x, int y) {
    if (is_ancestor(x, y))  // 如果 u 為 v 的祖先則 lca
        為 u
        return x;
    if (is_ancestor(y, x))  // 如果 v 為 u 的祖先則 lca
        為 u
        return y;

    // 判斷 2^logN, 2^(logN-1),...2^1, 2^0 倍祖先
    for (int i = logn; i >= 0; i--) {
        // cout << anc[x][i] << '\n';
        if (!is_ancestor(anc[x][i], y) &&
            anc[x][i])      // 如果 2^i 倍祖先不是 v 的
                祖先
            x = anc[x][i];  // 則往上移動
    }
    return anc[x][0];  // 回傳此點的父節點即為答案
}

// 以下是樹上差分
int tag[N], f[N], ans[N], root;
void add(int x, int y) {
    int lca = getlca(x, y);
    tag[x]++;
    tag[y]++;
    tag[lca]--;
    if (lca != root)
        tag[f[lca]]--;
}

int dfs_add(int now, int fa) {
    int diff = tag[now];
    for (auto nxt : g[now]) {
        if (nxt == fa)
            continue;
        diff += dfs_add(nxt, now);
    }
    return ans[now] = diff;
}
```

5.2 Eular Tour for LCA

```
#include <bits/stdc++.h>
using namespace std;
```

```
const int N = 1e5 + 5;

vector<int> tour, edge[N];
int timing = 0;
int in[N], out[N];

void dfs(int u, int fa) {
    tour.push_back(u);  // 進入的時間點
    in[u] = tour.size() - 1;
    for (auto v : edge[u]) {
        if (v == fa)
            continue;
        dfs(v, u);
        tour.push_back(u);  // 走完其中一個孩子 再走回自己
    }
    out[u] = tour.size() - 1;
}

void build(vector<int>& tour) {  // 把tour丟進線段樹初始化
    seg.build(tour);  // 除了tour，還要建每個點的深度
}

int query(int x, int y) {
    if (is_ancster(x, y))
        return x;  // LCA(x,y)=x
    if (is_ancster(y, x))
        return y;  // LCA(x,y)=y;
    if (out[y] < in[x])
        swap(x, y);  // 如果「y的區間」在「x的區間」的左邊，就交換
    int index =
        seg.query(out[x], in[y]);  // 取得區間內哪個位置的深度是最低的
    return tour[index];
}
```

5.3 DSU on Tree

```
int find(int x) {
    if (f[x] == x)
        return x;
    else
        return f[x] = find(f[x]);
}

void merge(int x, int y) {
    x = find(x), y = find(y);
    if (sz[x] < sz[y])
        swap(x, y);
    sz[x] += sz[y];
    f[y] = x;
}
void init() {
    // 初始化一開始大小都為1 (每群一開始都是獨立一個)
    for (int i = 1; i <= n; i++)
        sz[i] = 1;
    for (int i = 1; i <= n; i++)
        f[i] = i;
}
```

6 String Algorithms

6.1 Suffix Array (SAIS)

```
// 此為後綴陣列模板
// 用法:
// 把要處理的字串轉成整數陣列傳入suffix_array()，會得到SA與H維結果
const int N = 500010;  // 有可能要在這就寫原本字串的兩倍長
struct SA {
#define REP(i, n) for (int i = 0; i < int(n); i++)
#define REP1(i, a, b) for (int i = (a); i <= int(b); i++)
    bool _t[N * 2];
    int _s[N * 2], _sa[N * 2], _c[N * 2], x[N], _p[N],
        _q[N * 2],
        hei[N], r[N];
    int operator[](int i) {
        return _sa[i];
    }
    void build(int *s, int n, int m) {
        memcpy(_s, s, sizeof(int) * n);
        sais(_s, _sa, _p, _q, _t, _c, n, m);
        mkhei(n);
    }
    void mkhei(int n) {
        REP(i, n) r[_sa[i]] = i;
        hei[0] = 0;
        REP(i, n) if (r[i]) {
            int ans = i > 0 ? max(hei[r[i - 1]] - 1, 0)
                            : 0;
            while (_s[i + ans] == _s[_sa[r[i] - 1] +
                ans])
                ans++;
            hei[r[i]] = ans;
        }
    }
    void sais(int *s, int *sa, int *p, int *q, bool *t,
         int *c, int n,
              int z) {
        bool uniq = t[n - 1] = true, neq;
        int nn = 0, nmxz = -1, *nsa = sa + n, *ns = s +
            n, lst = -1;
#define MS0(x, n) memset((x), 0, n * sizeof(*(x)))
#define MAGIC(XD)                                  \
    MS0(sa, n);                                    \
    memcpy(x, c, sizeof(int) * z);                 \
    XD;                                            \
    memcpy(x + 1, c, sizeof(int) * (z - 1));       \
    REP(i, n)                                      \
    if (sa[i] && !t[sa[i] - 1])                    \
        sa[x[s[sa[i] - 1]]++] = sa[i] - 1;         \
    memcpy(x, c, sizeof(int) * z);                 \
    for (int i = n - 1; i >= 0; i--)               \
        if (sa[i] && t[sa[i] - 1])                 \
            sa[--x[s[sa[i] - 1]]] = sa[i] - 1;
        MS0(c, z);
        REP(i, n) uniq &= ++c[s[i]] < 2;
        REP(i, z - 1) c[i + 1] += c[i];
        if (uniq) {
            REP(i, n) sa[--c[s[i]]] = i;
            return;
        }
        for (int i = n - 2; i >= 0; i--)
            t[i] = (s[i] == s[i + 1] ? t[i + 1] : s[i]
                < s[i + 1]);
        MAGIC(REP1(i, 1, n - 1) if (t[i] && !t[i - 1])
            sa[--x[s[i]]] =
                p[q[i] = nn++] = i);
        REP(i, n) if (sa[i] && t[sa[i]] && !t[sa[i] -
            1]) {
            neq = lst < 0 ||
                  memcmp(s + sa[i], s + lst,
                         (p[q[sa[i]] + 1] - sa[i]) *
                             sizeof(int));
            ns[q[lst = sa[i]]] = nmxz += neq;
        }
        sais(ns, nsa, p + nn, q + n, t + n, c + z, nn,
            nmxz + 1);
        MAGIC(for (int i = nn - 1; i >= 0; i--) sa[--x[
            s[p[nsa[i]]]]] =
                  p[nsa[i]]);
    }
} sa;
int H[N], SA[N];  // SA: 後綴陣列的length
                  // H[i]: SA[i]與SA[i -1]的共同子字串長
void suffix_array(int *ip, int len) {
    // should padding a zero in the back
    // ip is int array, len is array length
    // ip[0..n-1] != 0, and ip[len] = 0
    ip[len++] = 0;
    sa.build(ip, len, 128);
    for (int i = 0; i < len; i++) {
        H[i] = sa.hei[i + 1];
        SA[i] = sa._sa[i + 1];
    }
    // resulting height, sa array \in [0,len)
}
```

6.2 Z-algorithm

```
int z[MAXN];
void Z_value(const string& s) {
    // z[i] = lcp(s[1...],s[i...])
    int i, j, left, right, len = s.size();
    left = right = 0;
    z[0] = len;
    for (i = 1; i < len; i++) {
        j = max(min(z[i - left], right - i), 0);
        for (; i + j < len && s[i + j] == s[j]; j++)
            ;
        z[i] = j;
        if (i + z[i] > right) {
            right = i + z[i];
            left = i;
        }
    }
}
```

6.3 Knuth-Morris-Pratt Algorithm

```
int failure[MXN];
vector<int> KMP(string& t, string& p) {
    vector<int> ret;  // 要保證長度 t > p
    for (int i = 1, j = failure[0] = -1; i < p.size();
        ++i) {
        while (j >= 0 && p[j + 1] != p[i])
            j = failure[j];
        if (p[j + 1] == p[i])
            j++;
        failure[i] = j;
    }
    for (int i = 0, j = -1; i < t.size(); ++i) {
        while (j >= 0 && p[j + 1] != t[i])
            j = failure[j];
        if (p[j + 1] == t[i])
            j++;
        if (j == p.size() - 1) {
            ret.push_back(i - p.size() + 1);
            j = failure[j];
        }
    }
    return ret;
}
```

6.4 Manacher's Algorithm

```
int ans[MAXN << 1];
char s[MAXN << 1];
string ret;  // 迴文字串儲存結果
void find_palindrome(char *s, int len, int *z) {
    int idx = 0, mx = -1;
    for (int i = 0; i < len; i++) {  // 找迴文半徑最大
        值
        if (mx < ans[i]) {
            idx = i;
            mx = ans[i];
        }
    }
    for (int i = idx - mx + 1; i < idx; i++)
        if (s[i] != '@')
            ret.push_back(s[i]);
    for (int i = idx; i <= idx + mx - 1; i++)
        if (s[i] != '@')
            ret.push_back(s[i]);
    // cout << ret << '\n'; //// 直接輸出
}
void z_value_pal(char *s, int len, int *z) {  // 字串，
    長度，輸出的陣列
    len = (len << 1) + 1;
    for (int i = len - 1; i >= 0; i--)
        s[i] = i & 1 ? s[i >> 1] : '@';
    z[0] = 1;
    for (int i = 1, l = 0, r = 0; i < len; i++) {
        z[i] = i < r ? min(z[l + l - i], r - i) : 1;
        while (i - z[i] >= 0 && i + z[i] < len &&
               s[i - z[i]] == s[i + z[i]])
            ++z[i];
        if (i + z[i] > r)
            l = i, r = i + z[i];
    }
    find_palindrome(s, strlen(s), z);
}
```

6.5 Minimum Rotation

```
int min_rotation(string s) {
    int a = 0, N = s.size();
    s += s;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (a + j == i || s[a + j] < s[i + j]) {
                i += max(0, j - 1);
                break;
            }
            if (s[a + j] > s[i + j]) {
                a = i;
                break;
            }
        }
    }
    return a;
}
// rotate(begin(s),begin(s)+minRotation(s),end(s))
// 上面註解可以直接讓一個字串做 rotate ，得到字典序最小
    的 rotation
```

6.6 Rolling Hash

```
ll P1 = 75577, P2 = 12721, MOD = 998244353;
pii hashs[MAXN], hashs2[MAXN];
string s1, s2;
void build(string s, pii hash) {
    ll val1 = 0, val2 = 0;
    for (int i = 0; i < s.length(); i++) {
        val1 = (val1 * P1 % MOD + s[i]) % MOD;
        val2 = (val2 * P2 % MOD + s[i]) % MOD;
        hash[i] = {val1, val2};
    }
}
ll minus(ll a, ll b) {
    return (a - b + MOD) % MOD;
}
ll cnt_occur() {  // 檢查
    int n = s1.size(), k = s2.size();
    ll cnt = (hashs2[k - 1].x == hashs[k - 1].x &&
              hashs2[k - 1].y == hashs[k - 1].y);
    ll val1 = 0, val2 = 0;
    for (int i = 1, j = k; j < n; j++, i++) {
        val1 = minus(hashs[j].x, hashs[i - 1].x * fpow(
            P1, j - i + 1));
        val2 = minus(hashs[j].y, hashs[i - 1].y * fpow(
            P1, j - i + 1));
        if (hashs2[k - 1].x == val1 && hashs2[k - 1].y
            == val2)
            cnt++;
    }
}
```

7 Geometry

7.1 Basic Geometry Structure

```
int dcmp(ld x) {  // 處理小數點精度的東東
    if (abs(x) < eps)
        return 0;
    else
        return x < 0 ? -1 : 1;
}
struct Pt {
    ld x, y;
    Pt(ld _x = 0, ld _y = 0) : x(_x), y(_y) {}
    Pt operator+(const Pt &a) {
        return Pt(x + a.x, y + a.y);
    }
    Pt operator-(const Pt &a) {
        return Pt(x - a.x, y - a.y);
    }
    Pt operator*(const ld &a) {
        return Pt(x * a, y * a);
    }
    Pt operator/(const ld &a) {
```

```
        return Pt(x / a, y / a);
    }
    ld operator*(const Pt &a) {  // 內積
        return x * a.x + y * a.y;
    }
    ld operator^(const Pt &a) {  // 行列式(叉乘)
        return x * a.y - y * a.x;
    }
    bool operator<(const Pt &a) const {
        return x < a.x || (x == a.x && y < a.y);
    }
    friend ld cross(Point a, Point b, Point o) {
        // 行列式(叉乘)，存入三個點
        Point p1 = a - o, p2 = b - o;
        return p1.x * p2.y - p1.y * p2.x;
    }
    // return dcmp(x-a.x) < 0 || (dcmp(x-a.x) == 0 &&
        dcmp(y-a.y) < 0);
    // }
    bool operator==(const Pt &a) {
        return dcmp(x - a.x) == 0 && dcmp(y - a.y) ==
            0;
    }
};

ld norm2(const Pt &a) {  // 平方
    return a * a;
}
ld norm(const Pt &a) {
    return sqrt(norm2(a));
}
Pt perp(const Pt &a) {  // 與原本向量垂直的向量
    return Pt(-a.y, a.x);
}
Pt rotate(const Pt &a, ld ang) {  // 旋轉矩陣
    return Pt(a.x * cos(ang) - a.y * sin(ang),
              a.x * sin(ang) + a.y * cos(ang));
}

struct Line {
    Pt s, e, v;  // start, end, end-start
    ld ang;
    Line(Pt _s = Pt(0, 0), Pt _e = Pt(0, 0)) : s(_s), e
        (_e) {
        v = e - s;
        ang = atan2(v.y, v.x);
    }
    bool operator<(const Line &L) const {
        return ang < L.ang;
    }
};

struct Circle {
    Pt o;
    ld r;
    Circle(Pt _o = Pt(0, 0), ld _r = 0) : o(_o), r(_r)
        {}
};
```

7.2 Circle Cover Area

```
struct CircleCover {
    int C;
    Circle c[N];  // 填入C(圓數量),c(圓陣列)
    bool g[N][N], overlap[N][N];
    // Area[i] : area covered by at least i circles
    D Area[N];
    void init(int _C) {
        C = _C;
    }
    bool CCinter(Circle &a, Circle &b, Pt &p1, Pt &p2)
        {
        Pt o1 = a.o, o2 = b.o;
        D r1 = a.r, r2 = b.r;
        if (norm(o1 - o2) > r1 + r2)
            return {};
        if (norm(o1 - o2) < max(r1, r2) - min(r1, r2))
            return {};
        D d2 = (o1 - o2) * (o1 - o2);
        D d = sqrt(d2);
        if (d > r1 + r2)
            return false;
        Pt u = (o1 + o2) * 0.5 +
               (o1 - o2) * ((r2 * r2 - r1 * r1) / (2 *
                   d2));
        D A = sqrt((r1 + r2 + d) * (r1 - r2 + d) * (r1
            + r2 - d) *
                   (-r1 + r2 + d));
        Pt v = Pt(o1.y - o2.y, -o1.x + o2.x) * A / (2 *
             d2);
        p1 = u + v;
        p2 = u - v;
        return true;
    }
    struct Teve {
        Pt p;
        D ang;
        int add;
        Teve() {}
        Teve(Pt _a, D _b, int _c) : p(_a), ang(_b), add
            (_c) {}
        bool operator<(const Teve &a) const {
            return ang < a.ang;
        }
    } eve[N * 2];
    // strict: x = 0, otherwise x = -1
    bool disjuct(Circle &a, Circle &b, int x) {
        return dcmp(norm(a.o - b.o) - a.r - b.r) > x;
    }
    bool contain(Circle &a, Circle &b, int x) {
        return dcmp(a.r - b.r - norm(a.o - b.o)) > x;
    }
    bool contain(int i, int j) {
        /* c[j] is non-strictly in c[i]. */
        return (dcmp(c[i].r - c[j].r) > 0 ||
                (dcmp(c[i].r - c[j].r) == 0 && i < j))
                    &&
               contain(c[i], c[j], -1);
    }
    void solve() {
        for (int i = 0; i <= C + 1; i++)
            Area[i] = 0;
        for (int i = 0; i < C; i++)
            for (int j = 0; j < C; j++)
                overlap[i][j] = contain(i, j);
        for (int i = 0; i < C; i++)
            for (int j = 0; j < C; j++)
                g[i][j] = !(overlap[i][j] || overlap[j
                    ][i] ||
                            disjuct(c[i], c[j], -1));
        for (int i = 0; i < C; i++) {
            int E = 0, cnt = 1;
            for (int j = 0; j < C; j++)
                if (j != i && overlap[j][i])
                    cnt++;
            for (int j = 0; j < C; j++)
                if (i != j && g[i][j]) {
                    Pt aa, bb;
                    CCinter(c[i], c[j], aa, bb);
                    D A = atan2(aa.y - c[i].o.y, aa.x -
                         c[i].o.x);
                    D B = atan2(bb.y - c[i].o.y, bb.x -
                         c[i].o.x);
                    eve[E++] = Teve(bb, B, 1);
                    eve[E++] = Teve(aa, A, -1);
                    if (B > A)
                        cnt++;
                }
            if (E == 0)
                Area[cnt] += pi * c[i].r * c[i].r;
            else {
                sort(eve, eve + E);
                eve[E] = eve[0];
                for (int j = 0; j < E; j++) {
                    cnt += eve[j].add;
                    Area[cnt] += (eve[j].p ^ eve[j +
                        1].p) * 0.5;
                    D theta = eve[j + 1].ang - eve[j].
                        ang;
                    if (theta < 0)
                        theta += 2.0 * pi;
                    Area[cnt] +=
                        (theta - sin(theta)) * c[i].r *
                             c[i].r * 0.5;
```

```
                }
            }
        }
    }
} ret;
```

7.3 Circles' Tangent Lines

```
vector<Line> go(const Circle& c1, const Circle& c2, int
     sign1) {
    // sign1 = 1 for outer tang, -1 for inter tang
    vector<Line> ret;
    double d_sq = norm2(c1.o - c2.o);
    if (d_sq < eps)
        return ret;
    double d = sqrt(d_sq);
    Pt v = (c2.o - c1.o) / d;
    double c = (c1.r - sign1 * c2.r) / d;
    if (c * c > 1)
        return ret;
    double h = sqrt(max(0.0, 1.0 - c * c));
    for (int sign2 = 1; sign2 >= -1; sign2 -= 2) {
        Pt n = {v.x * c - sign2 * h * v.y, v.y * c +
            sign2 * h * v.x};
        Pt p1 = c1.o + n * c1.r;
        Pt p2 = c2.o + n * (c2.r * sign1);
        if (fabs(p1.x - p2.x) < eps and fabs(p1.y - p2.
            y) < eps)
            p2 = p1 + perp(c2.o - c1.o);
        ret.push_back({p1, p2});
    }
    return ret;
}
```

7.4 Circles' Intersections

```
vector<Pt> interCircle(Pt o1, ld r1, Pt o2, ld r2) {
    if (norm(o1 - o2) > r1 + r2)
        return {};
    if (norm(o1 - o2) < max(r1, r2) - min(r1, r2))
        return {};
    ld d2 = (o1 - o2) * (o1 - o2);
    ld d = sqrt(d2);
    if (d > r1 + r2)
        return {};
    Pt u =
        (o1 + o2) * 0.5 + (o1 - o2) * ((r2 * r2 - r1 *
            r1) / (2 * d2));
    ld A = sqrt((r1 + r2 + d) * (r1 - r2 + d) * (r1 +
        r2 - d) *
                (-r1 + r2 + d));
    Pt v = Pt(o1.y - o2.y, -o1.x + o2.x) * A / (2 * d2)
        ;
    return {u + v, u - v};
}
```

7.5 Convex Hull

```
double cross(Pt o, Pt a, Pt b) {
    return (a - o) ^ (b - o);
}
vector<Pt> convex_hull(vector<Pt> pt) {
    sort(pt.begin(), pt.end());
    int top = 0;
    vector<Pt> stk(2 * pt.size());
    for (int i = 0; i < (int)pt.size(); i++) {
        while (top >= 2 &&
               cross(stk[top - 2], stk[top - 1], pt[i])
                   <= 0)
            top--;
        stk[top++] = pt[i];
    }
    for (int i = pt.size() - 2, t = top + 1; i >= 0; i
        --) {
        while (top >= t &&
               cross(stk[top - 2], stk[top - 1], pt[i])
                   <= 0)
            top--;
        stk[top++] = pt[i];
    }
    stk.resize(top - 1);
    return stk;
}
```

7.6 Convex Hull Tricks

```
struct Convex {
    int n;
    vector<Pt> A, V, L, U;
    Convex(const vector<Pt> &_A) : A(_A), n(_A.size())
        {  // n >= 3
        rotate(_A.begin(), min_element(all(_A)), _A.end
            ());
        // 最左的的點排最前面 有時可以不需要這一行
        auto it = max_element(all(A));
        L.assign(A.begin(), it + 1);
        U.assign(it, A.end()), U.push_back(A[0]);
        for (int i = 0; i < n; i++) {
            V.push_back(A[(i + 1) % n] - A[i]);
        }
    }
    int PtSide(Pt p, Line L) {
        return dcmp((L.b - L.a) ^ (p - L.a));
    }
    int inside(Pt p, const vector<Pt> &h, auto f) {
        auto it = lower_bound(all(h), p, f);
        if (it == h.end())
            return 0;
        if (it == h.begin())
            return p == *it;
        return 1 - dcmp((p - *prev(it)) ^ (*it - *prev(
            it)));
    }
    // 1. whether a given point is inside the CH
    // ret 0: out, 1: on, 2: in
    int inside(Pt p) {
        return min(inside(p, L, less{}), inside(p, U,
            greater{}));
    }
    static bool cmp(Pt a, Pt b) {
        return dcmp(a ^ b) > 0;
    }
    // 2. Find tangent points of a given vector
    // ret the idx of far/closer tangent point
    int tangent(Pt v, bool close = true) {
        assert(v != Pt{});
        auto l = V.begin(), r = V.begin() + L.size() -
            1;
        if (v < Pt{})
            l = r, r = V.end();
        if (close)
            return (lower_bound(l, r, v, cmp) - V.begin
                ()) % n;
        return (upper_bound(l, r, v, cmp) - V.begin())
            % n;
    }
    // 3. Find 2 tang pts on CH of a given outside
        point
    // return index of tangent points
    // return {-1, -1} if inside CH
    array<int, 2> tangent2(Pt p) {
        array<int, 2> t{-1, -1};
        if (inside(p) == 2)
            return t;
        if (auto it = lower_bound(all(L), p);
            it != L.end() and p == *it) {
            int s = it - L.begin();
            return {(s + 1) % n, (s - 1 + n) % n};
        }
        if (auto it = lower_bound(all(U), p, greater{})
            ;
            it != U.end() and p == *it) {
            int s = it - U.begin() + L.size() - 1;
            return {(s + 1) % n, (s - 1 + n) % n};
        }
        for (int i = 0; i != t[0]; i = tangent((A[t[0]
            = i] - p), 0))
            ;
        for (int i = 0; i != t[1]; i = tangent((p - A[t
            [1] = i]), 1))
            ;
        return t;
    }
    int find(int l, int r, Line L) {
        if (r < l)
            r += n;
```

```
        int s = PtSide(A[l % n], L);
        return *ranges::partition_point(views::iota(l,
            r), [&](int m) {
            return PtSide(A[m % n], L) == s;
        }) - 1;
    }
    // 4. Find intersection point of a given line
    // intersection is on edge (i, next(i))
    vector<int> intersect(Line L) {
        int l = tangent(L.a - L.b), r = tangent(L.b - L
            .a);
        if (PtSide(A[l], L) == 0)
            return {l};
        if (PtSide(A[r], L) == 0)
            return {r};
        if (PtSide(A[l], L) * PtSide(A[r], L) > 0)
            return {};
        return {find(l, r, L) % n, find(r, l, L) % n};
    }
};
```

7.7 Pick's Theory

```
vector<Pt> p;
ll compute_area(int n) {  // A * 2
    ll area = 0;
    for (int i = 1; i < n; i++)
        area += cross(p[i], p[(i + 1) % n], p[0]);
    area = abs(area);
    return area;
}
ll compute_pt_on_line(int n) {  // b
    ll pt = 0;
    for (int i = 0; i < n; i++)
        pt += __gcd(abs(p[i].x - p[(i + 1) % n].x),
                    abs(p[i].y - p[(i + 1) % n].y));
    return pt;  // 這是網路上找的
}
ll compute_inner_pt(int n) {  // i
    return (compute_area(n) - compute_pt_on_line(n)) /
        2 + 1;
}
```

8 Sorting

8.1 Merge Sort

```
template <typename T>
void merge_sort(T arr, T tmp, int l, int r) {
    // 陣列元素少於 1 直接結束
    if (r <= l + 1)
        return;

    // 分成左右兩邊處理
    int m = l + (r - l) / 2;
    merge_sort(arr, tmp, l, m);  // [l, m)
    merge_sort(arr, tmp, m, r);  // [m, r)

    // 合併兩邊的結果
    for (int i = l, li = l, ri = m; i < r; i++) {
        // 只剩下右邊 || 兩邊都有剩，右邊比左邊小
        if (li == m || (ri != r && arr[ri] < arr[li]))
            tmp[i] = arr[ri++];
        // 只剩下左邊 || 兩邊都有剩，左邊比右邊小
        else
            tmp[i] = arr[li++];
    }

    // 把結果複製回來
    for (int i = l; i < r; i++)
        arr[i] = tmp[i];
}
```

8.2 Quick Sort

```
template <typename T>
void quick_sort(T arr[], int l, int r) {
    // 陣列元素少於 1 直接結束
    if (r <= l + 1)
        return;

    // 左邊和右邊的指針
    int li = l + 1, ri = r - 1;
    // 判定的基準點
    T pivot = arr[l];

    // TODO: 左邊 <= 基準點 < 右邊
    while (li != ri) {
        // 右邊的指針往左移動，找比基準點小的值
        while (arr[ri] > pivot && li < ri)
            ri--;
        // 左邊的指針往右移動，找比基準點大的值
        while (arr[li] <= pivot && li < ri)
            li++;
        // 當左右指針沒有相遇時，表示不滿足 TODO
        if (li < ri)
            swap(arr[li], arr[ri]);
    }

    // 將基準點移到指針相遇的地方
    arr[l] = arr[li];
    arr[li] = pivot;

    quick_sort(arr, l, li);      // 處理基準點的左邊
    quick_sort(arr, li + 1, r);  // 處理基準點的右邊
}
```

9 Other Useful Shit

9.1 Discretization

```
int arr[MXN], tmp[MXN], n;
// arr[i] 是初始陣列, 長度為n, 且 0-base
void discretization() {
    for (int i = 0; i < n; i++)  // 將 arr 複製到 tmp
        tmp[i] = arr[i];
    sort(tmp, tmp + n);                          // 排序
    int len = unique(tmp, tmp + n) - (tmp);  // 把 tmp
        裡重複的數字去掉
    for (int i = 0; i < n; i++)
        arr[i] = lower_bound(tmp, tmp + len, arr[i]) -
            tmp;  // 二分搜
}
```

9.2 The Most Useful One

```
//                  !#########       #
//                !########!          ##!
//              !########!              ###
//           !##########                 ####
//         ######### #####               ######
//          !###!      !####!              ######
//            !          #####             ######!
//                         !####!          #######
//                            #####        #######
//                              !####!   #######!
//                                 ####!########
//               ##                  ##########
//             ,######!          !##############
//           ,#### ########################!####!
//         ,####'     ###################!'    #####
//       ,####'           #######               !####!
//     ####'                                       #####
//     ~##                                           ##~
//                    同志保佑 永無BUG

// ##            ##################
// ##            ##
// ##            ##
// ##            ##
// ##            ##
// ##            ##
// ##            ##
// ##            ##
// ################################
//               ##              ##
//               ##              ##
//               ##              ##
//               ##              ##
//               ##              ##
//               ##              ##
//               ##              ##
```

```
// ################                ##
//
//        元首保佑 永無BUG

//                    __
// _____ __ __      < @ \     _  __ _____
// \____\\_\\_)    /**|    (_//_//____/
//  \____\(_)\*\/****\/*/(_)/____/
//   \____\_/_\*\****/*/_\_/____/
//          u/u/u|****|u\u\u
//           u/u/|****|\u\u
//               |*||*|
//                |  |
//               /+--+\
//              || 卐 ||
//               \\==//
//
//          神獸保佑 永無BUG
```